

EXPERIMENT-14 Alpha-Beta Pruning Gaming

Aim

To implement the **Alpha-Beta pruning algorithm** in Python to optimize the Minimax search.

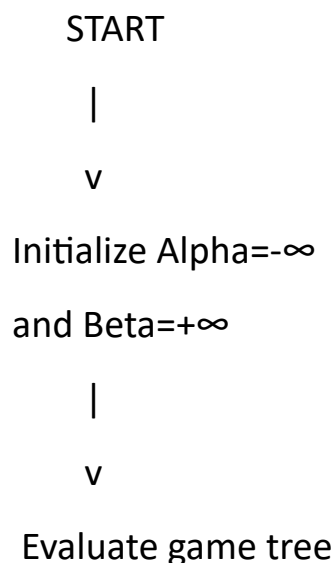
Objectives

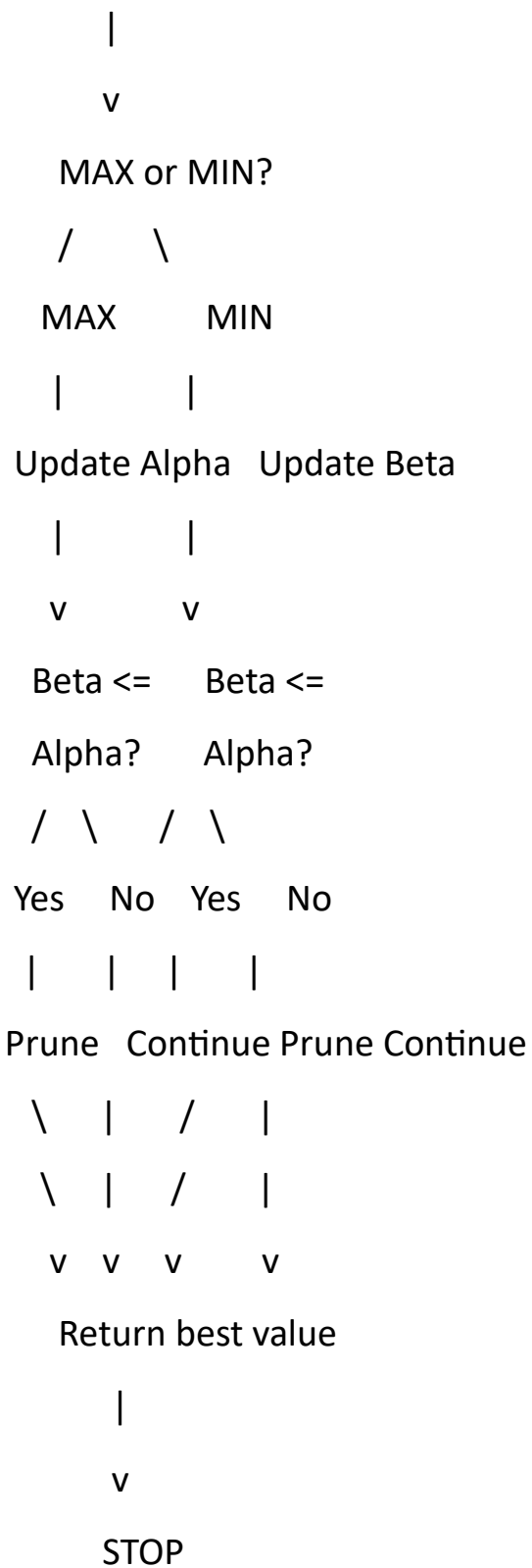
- To understand Alpha-Beta pruning.
- To reduce unnecessary search.
- To improve the efficiency of the Minimax algorithm.

Algorithm

1. Start with $\alpha = -\infty$ and $\beta = +\infty$.
2. Generate possible game states.
3. For MAX, update α .
4. For MIN, update β .
5. If $\beta \leq \alpha$, prune the remaining branches.
6. Continue until the optimal value is obtained.
7. Display the result.

Flowchart





Python Program

```
def alphabeta(depth, node_index, maximizing_player,
              values, alpha, beta):

    if depth == 3:
        return values[node_index]

    if maximizing_player:
        best = float('-inf')

        for i in range(2):
            value = alphabeta(
                depth + 1,
                node_index * 2 + i,
                False,
                values,
                alpha,
                beta
            )

            best = max(best, value)
            alpha = max(alpha, best)

        if beta <= alpha:
            print("Branch pruned at depth", depth)
```

```
break
```

```
return best
```

```
else:
```

```
best = float('inf')
```

```
for i in range(2):
```

```
    value = alphabeta(  
        depth + 1,  
        node_index * 2 + i,  
        True,  
        values,  
        alpha,  
        beta  
    )
```

```
best = min(best, value)
```

```
beta = min(beta, best)
```

```
if beta <= alpha:
```

```
    print("Branch pruned at depth", depth)
```

```
    break
```

```
return best
```

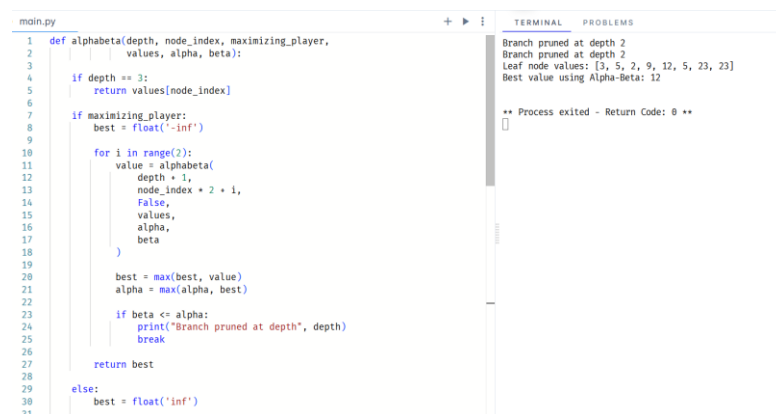
```
values = [3, 5, 2, 9, 12, 5, 23, 23]
```

```
result = alphabeta(  
    0, 0, True, values,  
    float('-inf'), float('inf')  
)
```

```
print("Leaf node values:", values)
```

```
print("Best value using Alpha-Beta:", result)
```

output:



```
1 def alphabeta(depth, node_index, maximizing_player,  
2             values, alpha, beta):  
3  
4     if depth == 3:  
5         return values[node_index]  
6  
7     if maximizing_player:  
8         best = float('-inf')  
9  
10        for i in range(2):  
11            value = alphabeta(  
12                depth + 1,  
13                node_index * 2 + i,  
14                False,  
15                values,  
16                alpha,  
17                beta  
18            )  
19            best = max(best, value)  
20            alpha = max(alpha, best)  
21  
22            if beta <= alpha:  
23                print("Branch pruned at depth", depth)  
24                break  
25  
26        return best  
27  
28    else:  
29        best = float('inf')
```

```
31
```

```
Branch pruned at depth 2  
Branch pruned at depth 2  
Leaf node values: [3, 5, 2, 9, 12, 5, 23, 23]  
Best value using Alpha-Beta: 12  
  
** Process exited - Return Code: 0 **
```

Result

The Alpha-Beta algorithm successfully finds the optimal value while **pruning unnecessary branches**.

Conclusion

Thus, Alpha-Beta pruning successfully improves Minimax by avoiding branches that cannot affect the final decision.